

CUDA & GPU Programming Practice Test

Part 2 - Python CUDA, Matrix Multiplication & Memory Coalescing — Questions 16-30 of 60 — Answers & Explanations

16. What does the `@cuda.jit` decorator do in a Numba-based CUDA program?

- A. It converts the function into a CPU-only optimized function
- B. It marks a Python function to be compiled into a GPU kernel
- C. It automatically parallelizes any Python loop without GPU involvement
- D. It disables just-in-time compilation for that function

Correct Answer: B

Explanation: The `@cuda.jit` decorator tells Numba to compile the decorated function as a CUDA kernel that executes on the GPU.

17. Why might a developer choose CuPy over writing raw CUDA C kernels for many GPU tasks?

- A. CuPy provides high-level, ready-to-use GPU-accelerated array operations with much less code
- B. Raw CUDA C is always slower than any Python-based approach
- C. CuPy requires no GPU driver to function
- D. CuPy cannot perform any GPU computation

Correct Answer: A

Explanation: For many common array operations, CuPy's NumPy-like interface lets developers get GPU acceleration without manually writing and tuning custom kernels.

18. What is an advantage of writing custom CUDA kernels with Numba compared to only using high-level libraries like CuPy?

- A. It guarantees better performance in every possible case
- B. It allows fine-grained control over thread-level GPU logic for specialized or non-standard operations
- C. It eliminates the need for a CUDA-capable GPU
- D. It removes the need for any GPU memory management

Correct Answer: B

Explanation: Custom kernels give explicit control over per-thread behavior, which is valuable for specialized algorithms not well covered by pre-built library functions.

19. What is the purpose of a simple 'vector addition' example when learning CUDA C basics?

- A. It only demonstrates CPU-side programming
- B. It is the most complex CUDA operation possible
- C. It introduces core concepts like kernel launches, thread indexing, and memory transfers in a minimal example
- D. It replaces the need to learn about GPU memory

Correct Answer: C

Explanation: Vector addition is a classic first CUDA example because it clearly demonstrates the essential building blocks—kernel launch, indexing, and data transfer—without added complexity.

20. What is cuBLAS commonly used for in the context of matrix multiplication on NVIDIA GPUs?

- A. It provides a highly optimized, vendor-supplied library for linear algebra operations like matrix multiplication

- B. It converts CUDA code into CPU-only code
- C. It is a debugging tool for CUDA kernels
- D. It manages GPU driver installation

Correct Answer: A

Explanation: cuBLAS is NVIDIA's optimized BLAS (Basic Linear Algebra Subprograms) library, offering highly tuned implementations of operations such as matrix multiplication.

21. Why does a naive matrix multiplication kernel typically perform much worse than a cuBLAS-based implementation?

- A. cuBLAS uses highly optimized memory access patterns and hardware-specific tuning that naive kernels lack
- B. cuBLAS avoids using the GPU entirely
- C. Naive kernels always use more GPU memory than cuBLAS
- D. Naive kernels cannot run on any NVIDIA GPU

Correct Answer: A

Explanation: cuBLAS incorporates extensive low-level optimizations, such as tiling and memory coalescing tuned per architecture, that a straightforward naive kernel typically does not implement.

22. What is shared memory in CUDA, and why is it used in optimized matrix multiplication kernels?

- A. A type of memory only accessible by a single thread
- B. A fast, on-chip memory space shared among threads in the same block, used to reduce repeated global memory access
- C. Memory that persists after the kernel finishes execution
- D. A memory region shared between the CPU and GPU only

Correct Answer: B

Explanation: Shared memory is a low-latency, on-chip memory accessible by all threads within a block, and is used to cache reused data like matrix tiles, reducing global memory traffic.

23. What is the purpose of atomic operations in CUDA programming?

- A. To disable thread synchronization entirely
- B. To permanently lock a GPU kernel from further execution
- C. To safely update a shared memory location from multiple threads without race conditions
- D. To convert floating-point numbers into integers

Correct Answer: C

Explanation: Atomic operations ensure that read-modify-write sequences on shared data are completed without interference from other threads, preventing race conditions.

24. What is the general purpose of a reduction operation in parallel GPU programming?

- A. To increase the size of an array
- B. To convert a kernel into sequential CPU code
- C. To disable all but one thread in a block
- D. To combine many values (such as summing an array) into a single result using parallel cooperation among threads

Correct Answer: D

Explanation: Reduction algorithms efficiently combine large numbers of values, like computing a sum or maximum, by having threads cooperate in a tree-like or stepwise parallel fashion.

25. What does warp shuffling allow threads within a warp to do in CUDA?

- A. Permanently disable a subset of threads in the warp
- B. Change which GPU executes the warp
- C. Convert the warp into multiple separate blocks
- D. Exchange values directly between threads in the same warp without using shared memory

Correct Answer: D

Explanation: Warp shuffle instructions let threads within the same warp exchange register values directly, often enabling faster reductions and data exchange than going through shared memory.

26. What does 'occupancy' refer to in the context of CUDA GPU performance?

- A. The clock speed of the GPU relative to its rated maximum
- B. The percentage of GPU memory currently allocated
- C. The number of GPUs installed in a system
- D. The ratio of active warps on a Streaming Multiprocessor to the maximum number it could support

Correct Answer: D

Explanation: Occupancy measures how many warps are actively resident on an SM compared to its theoretical maximum, which can influence the GPU's ability to hide latency.

27. Why might higher occupancy not always guarantee better kernel performance?

- A. Occupancy has no relationship to kernel execution at all
- B. Performance also depends on factors like memory bandwidth usage and instruction-level parallelism, not occupancy alone
- C. Occupancy is the only factor that affects GPU performance
- D. Higher occupancy always guarantees maximum performance in every case

Correct Answer: B

Explanation: While useful for hiding latency, occupancy is just one of several factors; a kernel bottlenecked elsewhere (e.g., memory bandwidth) may not benefit further from higher occupancy.

28. What is a DRAM burst in the context of GPU memory access?

- A. A type of GPU overheating event
- B. A method for compressing textures in graphics rendering
- C. A sudden failure of the GPU's memory hardware
- D. A block of consecutive memory addresses that the memory system reads or writes together in a single access

Correct Answer: D

Explanation: DRAM operates efficiently when accessed in bursts of contiguous addresses; understanding burst size helps explain why coalesced access patterns improve performance.

29. Why is matrix transpose often used as a case study for memory coalescing in CUDA courses?

- A. Naive transpose implementations can easily produce non-coalesced memory access patterns, making optimization techniques clearly visible
- B. Transpose is the only operation affected by memory coalescing
- C. Matrix transpose cannot be implemented on a GPU
- D. Transpose operations never involve any memory access

Correct Answer: A

Explanation: A straightforward transpose kernel often reads or writes memory in a strided, non-contiguous way, making it an instructive example for demonstrating coalescing optimizations like using shared memory.

30. What is the general focus of exercises based on 'PMPP' (Programming Massively Parallel Processors) chapter problems?

- A. Learning exclusively about CPU multithreading
- B. Designing GPU hardware circuits
- C. Practicing database query optimization
- D. Applying core GPU parallel programming concepts, such as memory access patterns and kernel design, to structured practice problems

Correct Answer: D

Explanation: PMPP is a well-known textbook on massively parallel programming, and its chapter exercises reinforce concepts like memory hierarchies, thread organization, and kernel optimization.